RV College of Engineering

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection

MAJOR PROJECT REPORT
(CS481P)

Submitted by,

Nikhil Vasu	**1RV22CS128**
Rohith Biradar	**1RV22CS164**
Suraj Chanaveeragoudra	**1RV22CS211**

Under the guidance of

Dr. Nagaraja G.S
Professor
Dept. of CSE
RV College of Engineering

Sanjana Ravindra Otihal
Assistant Professor
Dept. of CSE
RV College of Engineering

In partial fulfillment for the award of degree of

Bachelor of Engineering
in
Computer Science and Engineering

Department of CSE
2025-26

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection

A Major Project Report (CS481P)

Submitted by,

Nikhil Vasu	**1RV22CS128**
Rohith Biradar	**1RV22CS164**
Suraj Chanaveeragoudra	**1RV22CS211**

Under the guidance of

Dr. Nagaraja G.S
Professor
Dept. of CSE
RV College of Engineering

Sanjana Ravindra Otihal
Assistant Professor
Dept. of CSE
RV College of Engineering

In partial fulfillment of the requirements for the degree of Bachelor of Engineering in

Computer Science and Engineering

2025-26

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

Department of Computer Science and Engineering

CERTIFICATE

Certified that the major project (CS481P) work titled ***Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection*** is carried out by **Nikhil Vasu** (**1RV22CS128**), **Rohith Biradar** (**1RV22CS164**) and **Suraj Chanaveeragoudra** (**1RV22CS211**) who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in **Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Guide	**Head of the Department**	**Dean CSE Cluster**	**Principal**
Dr. Nagaraja G.S	Dr. Shanta Rangaswamy	Dr. Ramakanth Kumar P	Dr. K. N. Subramanya

External Viva

Name of Examiners	**Signature with Date**
1.	
2.	

DECLARATION

We, **Nikhil Vasu**, **Rohith Biradar** and **Suraj Chanaveeragoudra** students of eighth semester B.E., Department of Computer Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Computer Science and Engineering** during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name	**Signature**
1. Nikhil Vasu(1RV22CS128)	
2. Rohith Biradar(1RV22CS164)	
3. Suraj Chanaveeragoudra(1RV22CS211)	

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Nagaraja G.S**, Professor, Department of CSE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel members **Dr. Nagaraja G.S**, Professor, Department of CSE, RVCE and **Sanjana Ravindra Otihal**, Assistant Professor, Department of CSE, RVCE for the valuable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinators **Dr. Chethana R Murthy**, Professor, Department of CSE and **Prof. Deepika Dash**, Assistant Professor, for their timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. Shanta Rangaswamy**, Professor and Head, Department of Computer Science and Engineering, RVCE for the support and encouragement.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

We thank all the teaching staff and technical staff of Computer Science and Engineering department, RVCE for their help.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

Trade-Based Money Laundering (TBML) has emerged as a significant challenge within modern financial systems due to the increasing complexity and scale of global trade transactions. Conventional Anti-Money Laundering (AML) systems primarily rely on isolated rule-based monitoring frameworks that are often ineffective in identifying hidden transactional dependencies and multi-hop laundering typologies across interconnected banking networks. To address these limitations, this work developed a privacy-preserving Federated Heterogeneous Graph Neural Network (HGNN)-based framework for real-time multi-class TBML detection across distributed institutional environments.

The proposed framework modeled accounts, trade entities, SWIFT transactions, and trade documents as heterogeneous financial graphs using Memgraph. A hybrid analytical architecture integrating Graph Attention Network (GAT)-based convolutional layers and Long Short-Term Memory (LSTM)-based temporal sequence modeling was employed to capture relational topology and evolving transactional behavior. Federated learning using the Federated Averaging (FedAvg) aggregation strategy enabled collaborative cross-bank model optimization without centralized transactional data sharing. The implemented workflow incorporated Apache Kafka-based transaction streaming, graph construction, federated fraud inference, Suspicious Transaction Report (STR) generation, and regulator-facing compliance synchronization using Python, PyTorch Geometric, FastAPI, PostgreSQL, Memgraph, and React.js.

Experimental evaluation across distributed banking clients demonstrated stable federated convergence with Fraud-class Precision values approaching 0.98, F1-Scores nearing 0.84, and Precision–Recall Area Under Curve (PR-AUC) values exceeding 0.92. Concurrent scalability evaluation additionally maintained dashboard retrieval latency near 156–162 ms and fraud-filtering response latency near 148–159 ms under increasing institutional workloads, thereby validating the operational suitability of the proposed framework for scalable and privacy-preserving TBML detection. The framework can be extended into a fully integrated banking compliance platform with automated risk response, transaction control, and regulatory workflows. These enhancements would transform it from a monitoring system into a complete financial crime prevention ecosystem.

CONTENTS

LIST OF FIGURES

RASHTREEYA SIKSHANA SAMITHI TRUST ®
RV
INSTITUTIONS

LIST OF TABLES

RASHTREEYA SIKSHANA SAMITHI TRUST
RV
INSTITUTIONS

Chapter 1

Implementation

CHAPTER 1

IMPLEMENTATION

This chapter describes the implementation of the FedGAT-TBML framework: real-time transaction streaming, heterogeneous graph construction, a hybrid HGNN–LSTM detector, federated learning, and a dashboard for compliance monitoring. The system is implemented using Apache Kafka, Memgraph, PostgreSQL, PyTorch Geometric, Flower, FastAPI, and Next.js to enable scalable, real-time TBML detection.

1.1 Programming Language and Framework Selection

The implementation primarily uses Python for backend services and model training (PyTorch + PyTorch Geometric — e.g., `GATv2Conv` for graph attention), Flower for federated orchestration (custom `FedAvg` strategy), and Apache Kafka for durable streaming. Frontend and UI services use Next.js, TypeScript and Tailwind; FastAPI provides REST endpoints. Memgraph stores heterogeneous transaction graphs and PostgreSQL holds audit logs and STR records. Services are containerized with Docker for reproducible deployment.

1.2 Platform Selection

The proposed TBML detection framework is developed within a Windows-based environment consisting of interconnected frontend monitoring services and backend analytical infrastructures. The frontend platform manages authentication workflows, dashboard visualization, STR generation, regulatory monitoring, and institutional transaction management, while the backend analytical environment performs transaction streaming, graph generation, fraud inference, and federated model synchronization operations.

The implementation utilizes Docker-based isolated runtime environments for Kafka services, graph databases, backend APIs, and federated learning modules to improve deployment consistency and distributed service orchestration. Memgraph is utilized for graph relationship management and transactional connectivity analysis, whereas PostgreSQL functions as the centralized synchronization database between analytical services and dashboard environments. Visual Studio Code and GitHub are utilized for collaborative development, source-code management, debugging, and version control operations

throughout the implementation lifecycle.

1.3 Code Conventions

The codebase follows concise, practical conventions to improve readability and maintenance. Python modules and functions use `snake_case`; classes and UI components use `PascalCase`. Examples of graph entities and relationships include `(:Account)`, `(:Transaction)`, `(:Document)` and `[:SENDS]`, `[:TO]`, `[:HAS_DOC]`. REST endpoints follow clear, resource-oriented naming (e.g., `/api/transactions/{id}/score`). Configuration is managed via `.env` files, logging uses structured JSON logs, and type hints plus docstrings are used where helpful.

1.3.1 Naming Conventions

Naming conventions within the proposed framework are designed to remain semantically meaningful and aligned with domain-specific financial monitoring operations. Python variables, backend utility functions, Kafka services, and transaction-processing modules follow `snake_case` conventions, whereas analytical classes, federated learning modules, and frontend components utilize `PascalCase` notation. REST API endpoints, transaction services, and graph-processing utilities are named according to their operational functionality to improve implementation readability and maintainability.

Graph entities, transactional relationships, and database schemas additionally follow consistent naming conventions across graph and relational database environments. Examples include graph entities such as `(:Account)`, `(:Transaction)`, and `(:Document)`, along with transactional relationships including `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]` utilized throughout graph construction and fraud analysis operations.

1.3.2 File Organization

The proposed TBML detection framework follows a modular project structure where frontend monitoring services and backend analytical environments are maintained as interconnected distributed applications. The frontend environment contains dashboard interfaces, authentication services, STR management workflows, API handlers, reusable UI components, and regulatory monitoring modules implemented using the Next.js App Router architecture and TypeScript-based service organization.

The backend analytical environment contains Kafka producers and consumers, graph construction services, Hybrid HGNN–LSTM model implementations, federated learning

clients, inference pipelines, and database synchronization utilities responsible for real-time fraud analysis and transaction monitoring operations. Shared configuration files, utility services, logging environments, and centralized database connectors are additionally organized according to functional responsibilities to improve scalability, maintainability, and distributed service coordination across the proposed framework.

1.3.3 Environment Configuration and Logging

Environment variables and runtime configurations are securely managed using centralized `.env` configuration files to prevent direct exposure of API credentials, Kafka broker addresses, database configurations, authentication tokens, and federated learning parameters within the codebase. Backend services including Kafka consumers, Memgraph connectors, PostgreSQL clients, and fraud inference pipelines are initialized during runtime to support centralized service configuration and distributed analytical synchronization.

Logging mechanisms are integrated throughout transaction streaming services, graph-processing environments, federated learning modules, backend APIs, and fraud inference pipelines to support runtime monitoring, debugging, audit traceability, and compliance management operations. Fraud classifications, transaction verdicts, confidence scores, STR generation events, and institutional monitoring activities are additionally logged within PostgreSQL environments for dashboard synchronization and regulatory investigation workflows.

1.4 Difficulties Encountered and Strategies Used to Tackle

Implementation challenges included noisy and imbalanced labels, large graph sizes and memory constraints, streaming latency, and privacy-preserving coordination across institutional clients. Mitigations applied in the implementation include class rebalancing and targeted data augmentation for scarce labels, subgraph sampling and mini-batching to control memory, Kafka buffering and asynchronous consumers to reduce end-to-end latency, and Flower with secure aggregation and containerized deployments (Docker) to isolate client environments while enabling federated training.

1.5 System Implementation

The system implementation of the proposed TBML detection framework follows a distributed event-driven architecture integrating Kafka-based transaction streaming, heterogeneous graph generation, Hybrid HGNN–LSTM analytical processing, federated collaborative learning, real-time fraud inference, and dashboard-driven compliance monitoring. Backend analytical services, graph databases, inference pipelines, and frontend monitoring environments communicate through synchronized APIs, Kafka streams, and centralized PostgreSQL audit infrastructures to support scalable real-time Trade-Based Money Laundering detection and institutional monitoring operations.

1.5.1 Methodology of Synthetic Dataset Synthesis

The generation of the synthetic dataset is engineered to emulate the multi-modal complexities inherent in international trade finance. The process initializes by constructing a baseline network of financial entities—Accounts, Documents, and Transactions—governed by power-law distributions to simulate realistic institutional transaction volumes. Subsequent stages facilitate the injection of complex temporal patterns, commodity-specific price anomalies, and multi-currency layering effects, creating a high-fidelity environment for model training.

The underlying implementation utilizes a modular pipeline that leverages the Pandas library for vectorized data manipulation and Memgraph for graph-structured persistence. Each transaction record is anchored to its corresponding trade documentation through a schema-aligned bridge, ensuring that the structural connectivity required for Graph Neural Network (GNN) feature extraction is preserved. By integrating temporal constraints and noise-injection modules, the pipeline creates a realistic representation of financial behaviors, ranging from standard legitimate interactions to camouflaged, high-velocity laundering topologies.

1.5.2 Adversarial Refinement for Dataset Fidelity

To improve the realism of the synthetic dataset and reduce unintended learning biases, the data-generation pipeline was refined using multiple adversarial and statistically grounded modifications [1]. These refinements were introduced to ensure that the fraud-detection framework learned meaningful transactional and structural behaviors instead of relying on simple heuristic shortcuts or deterministic patterns.

1. **Price Multiplier Dynamics:** Static pricing rules were replaced with Gaussian Mixture Models (GMMs) and stochastic blending strategies so that a portion of fraudulent trades closely resembled legitimate market pricing behavior, thereby reducing simple threshold-based detection.

2. **Elimination of Feature Bias:** The *port_log_status* attribute was removed to avoid direct label leakage and encourage the Graph Neural Network to learn relational and structural dependencies more holistically.

3. **KYC/Dormancy Engineering:** Overlapping dormancy periods and controlled noise injection were introduced to generate more realistic customer activity distributions rather than binary dormant/active identifiers.

4. **Universal Topology Injection:** Complex graph structures such as Cycles, Fan-In, and Gather-Scatter patterns were distributed across all transaction classes to prevent structural bias where certain graph shapes directly imply fraud.

5. **Refined Pattern Classification:** Explicit pattern tags such as LEGIT and SUSPICIOUS were removed, and a neutral *No_Pattern* category was introduced to separate structural topology from fraud labeling.

6. **Temporal Realism:** The dataset generation process was aligned to a realistic one-year operational timeline (Dec 2024–Dec 2025) with transaction durations modeled according to practical financial activity windows.

7. **Scale-Free Network Topology:** Pareto/Zipf-based node distributions were introduced to simulate high-volume institutional or "Whale" entities capable of masking suspicious transactional behavior within dense financial activity.

8. **Log-Normal Transaction Amounts:** Transaction values were generated using Log-Normal distributions to better reflect real-world financial transaction behavior instead of uniformly distributed synthetic amounts.

9. **Multi-Hop Structural Integrity:** Rule-based generation constraints were enforced across multi-hop transaction paths to maintain graph consistency and realistic financial relationship propagation.

10. **FX Settlement/Currency-Jump Awareness:** Multi-currency transactional flows and FX spread variations were incorporated to expose the framework to cross-border layering and TBML settlement patterns.

11. **Stochastic Trade Price Blending:** Illicit trade pricing distributions were blended with legitimate transactional behaviors to reduce detectable tabular shortcuts and encourage context-aware inference.

12. **Data Leakage Mitigation:** Auxiliary indicators capable of providing deterministic fraud identification were systematically removed to improve generalization during model training.

13. **Endurance-Based Simulation:** Multi-hop transactional cycles were modeled as sustained operational behaviors rather than isolated short-duration activities to improve structural realism within the generated financial graph.

1.5.3 Kafka-Based Transaction Streaming

The proposed TBML detection framework utilizes Apache Kafka to support asynchronous transaction streaming and event-driven communication between distributed analytical services. A Kafka producer continuously generates synthetic SWIFT transaction payloads containing sender identifiers, receiver identifiers, transaction amounts, commodity information, trade quantities, price deviation scores, and weight-gap attributes before publishing them into the `incoming_swift_messages` topic for downstream analytical processing.

Kafka consumer services continuously subscribe to the transaction stream and process incoming financial events in real time for graph generation and fraud inference operations. The consumer pipeline validates transaction payloads, constructs heterogeneous graph relationships within Memgraph, extracts multi-hop transactional subgraphs, and forwards graph tensors to the Hybrid HGNN–LSTM inference engine for downstream fraud prediction and PostgreSQL audit logging. The streaming architecture enables decoupled communication between frontend services, graph-processing environments, and distributed analytical modules operating within the proposed framework.

1.5.4 Graph Construction and Feature Extraction

The graph construction pipeline transforms streamed financial transactions into heterogeneous graph representations for downstream graph-based analytical processing. Incoming transaction payloads consumed from the `incoming_swift_messages` Kafka topic are processed using Cypher-based graph ingestion operations within Memgraph, where transactional entities and financial relationships are dynamically mapped into interconnected graph structures as illustrated in Figure 1.1.

The generated graph schema consists of heterogeneous node entities including `(:Account)`, `(:Transaction)`, and `(:Document)` connected through relationships such as `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]`. The feature extraction pipeline subsequently performs constrained multi-hop subgraph traversal around the target transaction to generate node feature tensors, graph edge mappings, sequential transaction representations, and trade metadata tensors required for Hybrid HGNN–LSTM-based fraud inference operations.

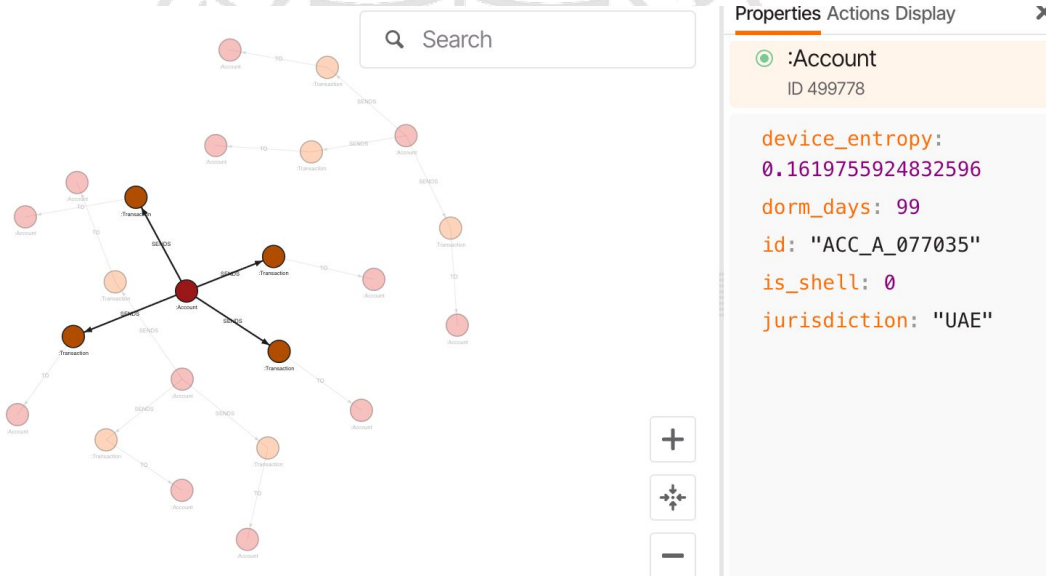

Figure 1.1: Heterogeneous financial transaction graph visualized within the Memgraph Lab environment.

1.5.5 Hybrid HGNN–LSTM Analytical Implementation

The proposed Hybrid HGNN–LSTM analytical module is implemented using PyTorch and PyTorch Geometric to perform graph-driven fraud analysis, temporal transaction modeling, and trade metadata learning for intelligent TBML detection. The implementation receives heterogeneous graph tensors generated from the Memgraph environment, including KYC node attributes, transactional edge relationships, sequential SWIFT trans-

action records, and trade-deviation features before forwarding them into the analytical inference pipeline. By combining graph attention learning with temporal sequence analysis and trade-feature fusion, the framework is capable of capturing both structural and behavioral fraud patterns across interconnected financial transaction networks.

The graph-learning branch utilizes a three-layer `GATv2Conv` architecture with multi-head attention mechanisms to perform neighborhood aggregation and graph message passing across interconnected financial entities [2]. The implemented graph layers process KYC-related node attributes such as shell-company indicators, jurisdiction metadata, and dormant-account characteristics to generate enriched graph embeddings capable of exposing hidden intermediary relationships and suspicious transaction-routing behavior. ReLU activation functions and dropout regularization are integrated after each graph convolution layer to improve representation stability during federated training operations. The generated graph embeddings are then aggregated using `global_mean_pool()` and fused with sequential SWIFT transaction features before being processed through the LSTM branch for temporal anomaly detection and sequential transaction behavior modeling. The temporal branch captures repeated payment structures, abnormal transaction timing behavior, and cyclic financial movement patterns that are difficult to identify using standalone graph-learning approaches.

Trade-specific metadata such as price-deviation attributes and cargo weight-gap indicators are independently processed using multilayer perceptron operations consisting of fully connected layers, Layer Normalization, ReLU activations, and dropout regularization mechanisms. The generated graph embeddings, temporal sequence representations, and trade-feature vectors are fused within the final analytical layer before being forwarded into the dense classification module for fraud prediction. The final classification layer categorizes transactions into predefined classes including `CLEAN`, `WATCHLIST`, and `FRAUD`. Experimental evaluation demonstrated stable federated convergence and strong fraud-class Precision–Recall performance across distributed institutional datasets. The overall implementation workflow of the proposed Hybrid HGNN–LSTM analytical module is illustrated in Figure 1.2.

```
# ========================================
# HYBRID HGNN--LSTM IMPLEMENTATION
# ========================================

# --- Graph Attention Layers ---
# Multi-hop neighborhood aggregation
self.gat1 = GATv2Conv(
    in_channels=kyc_dim, out_channels=hidden_dim, heads=2, concat=False
)

self.gat2 = GATv2Conv(
    in_channels=hidden_dim, out_channels=hidden_dim, heads=2, concat=False
)

self.gat3 = GATv2Conv(
    in_channels=hidden_dim, out_channels=hidden_dim, heads=2, concat=False
)

# --- Temporal Sequence Branch ---
# Sequential SWIFT transaction modeling
self.lstm = nn.LSTM(
    input_size=hidden_dim + swift_dim, hidden_size=lstm_hidden, batch_first=True
)

# ========================================
# GRAPH MESSAGE PASSING
# ========================================

x = F.dropout(F.relu(self.gat1(kyc_x, edge_index)), p=0.2, training=self.training)

x = F.dropout(F.relu(self.gat2(x, edge_index)), p=0.2, training=self.training)

enriched_nodes = F.dropout(
    F.relu(self.gat3(x, edge_index)), p=0.2, training=self.training
)
```

```
# ========================================
# GLOBAL GRAPH POOLING
# ========================================

graph_context = global_mean_pool(enriched_nodes, batch).unsqueeze(1)

# ========================================
# GRAPH + TEMPORAL FEATURE FUSION
# ========================================

graph_context_expanded = graph_context.expand(seq_data.size(0), seq_data.size(1), -1)

combined_seq = torch.cat([seq_data, graph_context_expanded], dim=-1)

# Temporal anomaly detection
lstm_out, _ = self.lstm(combined_seq)

# ========================================
# TRADE FEATURE PROCESSING
# ========================================

trade_emb = self.trade_mlp(trade_features)

# ========================================
# FINAL FRAUD CLASSIFICATION
# ========================================

fused_vector = torch.cat([final_lstm_step, trade_emb], dim=1)

return self.classifier(fused_vector)
```

Figure 1.2: Implementation workflow of the proposed Hybrid HGNN–LSTM analytical module.

1.5.6 Federated Learning Implementation

The federated learning environment is implemented using the Flower federated learning framework to support distributed privacy-preserving fraud detection across multiple institutional clients. The implementation utilizes a centralized aggregation server and three participating institutional clients, where each client performs localized training of the proposed Hybrid HGNN–LSTM analytical model using institution-specific transaction graphs, sequential SWIFT transaction representations, and fraud classification labels without directly sharing sensitive financial records outside the local analytical environment.

The centralized aggregation server implements a custom federated strategy by extending the `flwr.server.strategy.FedAvg` class and overriding the `aggregate_fit()` operation for distributed parameter aggregation. During each federated communication round, locally trained model weights generated from participating institutional clients are transmitted to the aggregation server where the `FedAvg` algorithm performs parameter averaging across synchronized client updates to generate a globally optimized fraud detection model. The server initializes the Hybrid HGNN–LSTM analytical architecture using synchronized feature dimensions including three-dimensional KYC node features, one-dimensional SWIFT transaction attributes, two-dimensional trade metadata features, hidden embedding dimensions of size 32, and three-class fraud prediction outputs.

The federated implementation performs ten synchronized communication rounds using complete client participation configurations including `fraction_fit=1.0`, `min_fit_clients=3`, and `min_available_clients=3` to maintain stable distributed convergence across institutional environments [3]. After the final aggregation round, the globally optimized model parameters are converted from Flower parameter streams into NumPy tensors using `parameters_to_ndarrays()` before being serialized and stored as `global_model.npz` for downstream real-time fraud inference operations. The implemented federated workflow enabled collaborative fraud learning, stable distributed convergence, and privacy-preserving analytical synchronization while maintaining complete institutional data isolation across participating financial organizations.

1.5.7 User Interface and Compliance Workflow Implementation

The user interface and compliance-monitoring environment is implemented using Next.js, TypeScript, Tailwind CSS, Prisma ORM, Zustand state management, and PostgreSQL to support secure institutional fraud monitoring and centralized regulatory investigation workflows. The frontend architecture follows a modular service-oriented design consisting of protected dashboard routes, middleware-based authentication guards, API handlers, Prisma-powered database interaction layers, and role-specific monitoring interfaces integrated with the federated analytical inference environment.

Authentication and Role Validation Pipeline

The authentication workflow is implemented using JWT-based session authorization combined with centralized middleware validation and protected route interception mechanisms. Incoming requests targeting protected dashboard routes are validated through middleware guards where session tokens are verified against predefined authorization scopes including `BANK_ADMIN` and `REGULATOR`. Once validated, the session context is propagated across frontend environments through centralized authentication providers and Zustand-managed session states before protected dashboard components and compliance services are rendered within the client environment.

Transaction Monitoring and Dashboard Rendering

Fraud predictions generated by the federated Hybrid HGNN–LSTM inference pipeline are synchronized into centralized PostgreSQL audit tables through backend transaction-processing services. Protected Next.js API handlers subsequently retrieve transaction

classifications, fraud scores, STR metadata, and institutional monitoring information using Prisma query operations before dynamically rendering analytical outputs within role-specific dashboard interfaces. The banking administrator dashboard supports transaction filtering, fraud investigation workflows, watchlist escalation procedures, and false-positive reclassification operations through synchronized API-driven transaction state updates across centralized audit environments.

STR Generation and Notification Workflow

Transactions classified as fraudulent trigger Suspicious Transaction Report (STR) generation workflows implemented through secured API endpoints and PostgreSQL-backed audit synchronization services. Institutional administrators initiate STR generation requests through protected dashboard interfaces where transaction metadata, fraud classifications, investigation summaries, institutional remarks, and compliance status information are serialized and stored within centralized STR audit tables. The generated STR records are subsequently propagated into regulator-facing monitoring environments through synchronized notification services and backend compliance-routing workflows.

Regulatory Investigation and Compliance Monitoring

The regulatory monitoring environment consumes institution-generated STR records, fraud-classified transaction histories, compliance metadata, and supporting analytical evidence through protected API routes and Prisma-based database interaction layers. Regulatory authorities can perform investigation workflows including document-verification requests, institutional inquiry procedures, transaction review operations, and account-freeze initiation requests through centralized compliance interfaces synchronized with backend audit environments. The implemented monitoring workflow supports synchronized fraud prediction, institutional verification, STR lifecycle management, and centralized compliance monitoring within a unified real-time TBML analytical platform.

1.6 Summary

This chapter presented the implementation workflow of the proposed TBML detection framework including Kafka-based transaction streaming, heterogeneous graph construction, Hybrid HGNN–LSTM analytical processing, federated collaborative learning, real-time fraud inference, and dashboard-driven compliance monitoring. The chapter discussed the implementation of distributed analytical services, graph-processing pipelines,

federated parameter aggregation mechanisms, inference synchronization workflows, and centralized PostgreSQL audit environments operating within the proposed framework.

The implementation chapter further demonstrated how graph attention learning, temporal transaction modeling, trade feature fusion, federated analytical synchronization, and role-based compliance monitoring are integrated within a unified real-time TBML detection environment. The subsequent chapter presents the experimental evaluation, performance analysis, federated convergence results, and monitoring outputs generated by the proposed TBML detection framework.

RASHTREEYA SIKSHANA SAMITHI TRUST ® RV INSTITUTIONS

Appendix A

Plagiarism report

APPENDIX A

PLAGIARISM REPORT

Attach the screenshot of plagiarism report front page.

RASHTREEYA SIKSHANA SAMITHI TRUST ®
RV
INSTITUTIONS

Appendix B

Publication details

APPENDIX B

PUBLICATION DETAILS

Attach the screenshot of either the Acknowledgment of the publication / Proof of Acceptance / Proof of Registration.

RASHTREEYA SIKSHANA SAMITHI TRUST ® RV INSTITUTIONS

Appendix C

Code

APPENDIX C

CODE

C.1 First Appendix

You can use `tcblisting` for creating the code snippets. The following example illustrates how one can customize the `tcblisting` to achieve the `tcl` script. Similarly, one can use it for other programming language listing, including HDL.

```
# Since our design has a clock with name clk,
## specify that name under [get_port ]
create_clock -period 40 -waveform {0 20} [get_ports clk]

# Setting a 'delay' on the clock:
set_clock_latency 0.3 clk

# Setting up constraints on your I/P and O/P pins
set_input_delay 2.0 -clock clk [all_inputs]
set_output_delay 1.65 -clock clk [all_outputs]

# Set realistic 'loads' on each output pin
set_load 0.1 [all_outputs]

# Set 'maximum' fanin and fan-out for the input and output pins
set_max_fanout 1 [all_inputs]
set_fanout_load 8 [all_outputs]
```

BIBLIOGRAPHY

[1] K. Sheka, K. Victor, and E. Walker, "A synthetic data generation framework for money laundering detection using behavioral analysis learning approach in mobile banking systems," *ResearchGate*, Dec. 2024.

[2] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" In *International Conference on Learning Representations (ICLR)*, 2022. arXiv: 2105.14491.

[3] C. H. Kennedy, A. Hilal, and M. Momeni, "The role of federated learning in improving financial security: A survey," in *IEEE Global Conference on Artificial Intelligence and Internet of Things (GCAIoT)*, IEEE, 2025, pp. 1–8. arXiv: 2510.14991.

RASHTREEYA SIKSHANA SAMITHI
RV